

This problem set is due by 11:59pm on **Wednesday, April 2**. It should be submitted through Canvas. All submissions should be typeset using LaTeX.

Problem 1 (Deep Q-Learning) In Lecture 9, we discussed fitted Q -learning. In this problem, we will explore Q -learning where our Q -function is represented by a deep neural network. There are some subtle implementation details associated with the Deep Q -learning approach not discussed in lecture. This problem will attempt to walk you through these details to control the simple pendulum.

- a) **(1 point)** In this problem, we will be leveraging the MATLAB Deep Learning Toolbox. Open the code `deep_qlearning.m`. In this problem, we will implement a deep Q -function for a discrete set of actions. Instead of using a deep neural network to implement a Q -function of the form $q = Q(x, u)$ where $x \in \mathbb{R}^{N_x}$, $u \in \mathbb{R}^{N_u}$, and $q \in \mathbb{R}$, we will implement a Q -function of the form $q_a = Q(x)$ where $q_a \in \mathbb{R}^{N_a}$. Here q_a is a vector of scalar quality value associated with each discrete action $a = \{a_0, a_1, \dots, a_{N_a}\}$. Fill in the number of nodes for each connected layer.
- b) **(3 points)** We must write functions to set and get the outputs of our deep Q -network. Consider the function `get_q_factor`. This function accepts a vector of actions ($a \in \mathbb{Z}^{N_a}$) and a matrix of Q -values ($Q_x \in \mathbb{R}^{N_s \times N_a}$) evaluated at some number of N_s states. This function should return a vector of Q -values $Q_a \in \mathbb{R}^{N_s}$ for each of the actions provided. Similarly, consider the function `set_q_factor`. This function replaces entries in a matrix of Q -values ($Q_x \in \mathbb{R}^{N_s \times N_a}$) evaluated at some number of N_s states using a vector of Q -values evaluated at some set of state/action pairs ($Q_a \in \mathbb{R}^{N_s}$). Implement these functions. We will also need to be able to compute the optimal Q -values a matrix of Q -values ($Q_x \in \mathbb{R}^{N_s \times N_a}$) evaluated at some number of N_s states. This should be implemented in `get_Qstar`.
- c) **(2 points)** A second major difference with the Deep Q -Learning algorithm is that two neural networks are used to represent the Q -function. The first, often referred to as the main network, is updated following each temporal difference update. This is the network which is trained or fit from data. It is denoted by Q_m in the code. The second network, often referred to as the target network, is used to complete the temporal difference update. It is denoted as Q_t in the code. This network is updated less often, at some predefined time period (e.g., every 200 steps), by copying over the weights from the main network. Using these two networks affords greater algorithmic stability. Complete the “train” function by inserted the correct network into the calls for `td_update` and `learn_q_factor`.
- d) **(3 points)** As in Q -learning for grid world, we must implement the temporal difference update. Fill in the function `td_update`. Also complete the `learn_q_factor` function.

- e) **(4 points)** Run the Deep Q -learning algorithm to generate a Q -function for the pendulum. Demonstrate that the algorithm can achieve the swing-up task. Submit your code and plots of the cost-to-go and swing-up trajectory. Expect to run the algorithm for at least 15-30 minutes to learn the Q -function.
- f) **(Extra Credit 10 points)** Compute the Q -function for the cart-pole system. Submit your code and relevant plots of executing the resulting policy.

Problem 2 (Final Project Proposal) (20 points) Please submit a 2-3 page proposal for your final project. The goal of the final project is to more deeply explore learning-based control applied to a robotics problem of your choice. The problem must involve a dynamical robotic system where learning is necessary to achieve the required desired performance. In your project proposal, be sure to include the following:

- A candidate dynamical robotic system.
- A description of an approximate physics-based model that captures the dynamics of the robotic system.
- A discussion of the major sources of uncertainty in the physical system and how the physical system may differ from the model.
- A description of the dataset or simulator to be used for developing algorithms and evaluating results.
- A proposed learning-based control approach and identification of the learning paradigm(s) to be used (and why).
- A discussion of the novelty of your approach and/or problem.
- The metric(s) used to determine success.
- A discussion of the major challenges that may be faced during the project.
- A short discussion of related work in the field and a list of references.
- A schedule with specific milestones.